



UiA University
of Agder

Group 7

ZTYPE

Author

Filmon Berhe

Iyad Zidan

Mohamed Ali Abdullahi

Yeronis Assefa Hubena

Project For DAT220 - Software Development 2

Supervised by Christian Auby

Faculty Of Engineering and Sciences

University of Agder

Grimstad, desember 2019

Abstract

Shooting games are popular among some friends of this group. But type to shoot games stands out among them. For us in this group, the chance to write a game similar to the one loved by our friends, presented itself as a final project in DAT220, a course in software development-2 at the University of Agder. This report will take the reader through the development process to our game, Type to shoot.

Our game is inspired by ZTYPE[7] developed by DOMINIC SZABLEWSKI[5] and sponsored by Typesy[6]. The game is to shoot every word by typing fast enough so that they don't crush to the weapon at the bottom of the window. The challenge is to deal with the higher intensity of words as the level increases and their pressure to crush the shooter. There are limited features implemented compared to the above mentioned game, as the game lacks some advanced animations and effects. The challenge is to acquire advanced technical skills in C++ by getting involved in developing an effective software as a group. It is meant to help us use our earlier technical knowledge in c++ and advance it more practically. More importantly, the project introduced us to challenges like version control, working in groups, project management and how software development works in reality. Being our first project in software development-2 and given the above challenges are unfamiliar to majority of the group, the game implements a simple shooting game with minimum requirements to be called a shooting game.

Obligatorisk gruppeerklæring

Den enkelte student er selv ansvarlig for å sette seg inn i hva som er lovlige hjelpemidler, retningslinjer for bruk av disse og regler om kildebruk. Erklæringen skal bevisstgjøre studentene på deres ansvar og hvilke konsekvenser fusk kan medføre. Manglende erklæring fritar ikke studentene fra sitt ansvar.

1.	Vi erklærer herved at vår besvarelse er vårt eget arbeid, og at vi ikke har brukt andre kilder eller har mottatt annen hjelp enn det som er nevnt i besvarelsen.	Ja
2.	Vi erklærer videre at denne besvarelsen: • Ikke har vært brukt til annen eksamen ved annen avdeling/universitet/høgskole innenlands eller utenlands. • Ikke refererer til andres arbeid uten at det er oppgitt. • Ikke refererer til eget tidligere arbeid uten at det er oppgitt. • Har alle referansene oppgitt i litteraturlisten. • Ikke er en kopi, duplikat eller avskrift av andres arbeid eller besvarelse.	Ja
3.	Vi er kjent med at brudd på ovennevnte er å betrakte som fusk og kan medføre annullering av eksamen og utestengelse fra universiteter og høyskoler i Norge, jf. Universitets- og høyskoleloven §§4-7 og 4-8 og Forskrift om eksamen §§ 31.	Ja
4.	Vi er kjent med at alle innleverte oppgaver kan bli plagiatkontrollert.	Ja
5.	Vi er kjent med at Universitetet i Agder vil behandle alle saker hvor det forligger mistanke om fusk etter høyskolens retningslinjer for behandling av saker om fusk.	Ja
6.	Vi har satt oss inn i regler og retningslinjer i bruk av kilder og referanser på biblioteket sine nettsider.	Ja
7.	Vi har i flertall blitt enige om at innsatsen innad i gruppen er merkbart forskjellig og ønsker dermed å vurderes individuelt. Ordinært vurderes alle deltakere i prosjektet samlet.	Nei

Publiseringsavtale

Fullmakt til elektronisk publisering av oppgaven Forfatter(ne) har opphavsrett til oppgaven. Det betyr blant annet enerett til å gjøre verket tilgjengelig for allmennheten (Åndsverkloven. §2).

Oppgaver som er unntatt offentlighet eller taushetsbelagt/konfidensiell vil ikke bli publisert.

Vi gir herved Universitetet i Agder en vederlagsfri rett til å gjøre oppgaven tilgjengelig for elektronisk publisering:	Nei
Er oppgaven båndlagt (konfidensiell)?	Nei
Er oppgaven unntatt offentlighet?	Nei

Contents

1	Introduction	1
1.1	Product Vision	1
1.2	Similar Products	2
2	Process	4
2.1	Preparation	4
2.2	Meetings	5
2.3	Tools	6
2.3.1	Jira	6
2.3.2	Bitbucket	6
2.3.3	Overleaf	7
2.3.4	Clion	7
2.3.5	Git	7
2.4	Libraries	7
2.5	Version control	8
2.6	Gameplay	9
2.6.1	Window overview	9
2.6.2	Game logic	9
2.7	MVP (Minimal viable product)	10
2.8	Division Of Labor	11
3	Product	13
3.1	Main menu	13
3.2	Weapon	14
3.3	Bullet	14
3.4	words	16

3.5	Storage	17
3.6	Scoreboard	17
4	Implementation	18
4.1	General Structure	18
4.2	Game State	20
4.3	Class Object	21
4.3.1	Class Entity	21
4.3.2	Waves of words	22
4.3.3	Isolate target (word)	23
4.3.4	Moving Entity towards the object	23
4.3.5	alphabet deletion in Entity class	24
4.3.6	EntityProxy class	24
4.4	Finding Direction	25
4.4.1	direction of Bullet	25
4.5	Event	27
4.6	Class Storage	28
4.6.1	Creating dynamic levels	28
4.7	GameManager class	29
4.8	MenuManager class	29
5	Testing and validating	32
6	Discussion	34
6.1	Process	34
6.2	Git	34
6.3	Product	35
6.4	Design and multimedia	35

6.5	Words overlapping	36
6.6	Rotating of bullet and weapon	36
6.7	Showing score and status	37
6.8	Challenges	37
6.8.1	Shooting movement	37
6.8.2	Multiple Bullets	38
7	Future plans	39
7.1	Load and save	39
7.2	Network	39
7.3	Pause and resume	39
7.4	Elimination of words	39
7.5	Explosion animation	40
7.6	Loading own text	40
8	conclusion	41
9	Individual Report	42
9.1	Filmon Berhe Mebrahtom	42
9.2	Iyad Zidan	43
9.3	Mohamed Ali Abdullahi	44
9.4	Yeronis Assefa Hubena	45

List of Figures

1	ZTYPE Type to shoot	3
2	Dividing the product implementation in modules	5
3	Discussed points during one of the meeting days	6
4	menu window	14
5	movement of bullet	15
6	Enemies spawning	16
7	Game components and their most important properties	18
8	Class Diagram	19
9	Game States	20
10	UML representation of Class Object	21
11	Object-Entity-EntityProxy relationship	22
12	Type shooting game Object	27
13	Storage - EntityProxy Association	28

List of Tables

1	Minimal Viable product	11
2	Test and their results	32
3	Strike-through shows implemented features	35

Listinger

1	state	20
2	Deleting alphabet implementation	23
3	Deleting alphabet implementation	24
4	distance	25

5	direction	26
6	menu window implementation	31

1 Introduction

This project is part of Software Development- 2 course in C++. In this course, students had the option of choosing between Game development and Embedded System. Our group have chosen game development, as all the members are interested in developing games. There are variety of games to play on the computer, from variety of fields such as entertainment, educational, business, etc.

1.1 Product Vision

The game chosen for this project is an educational yet entertaining game. Everyone who uses computers with keyboards, have to deal with typing and to accomplish work efficiently, it is preferable if one writes faster and alphabetically correct. Hence, we provided this game so that people can learn how to write grammatically correct sentences and increase their typing speed. There is no age limit to play this game. Everyone who is able to use computer, can play this game and learn a new skill. This product is for everyone (capable of typing), interested to spend some good time. Its not so complicated for children and not so simple for anyone who already knows how to type either.

This product is aimed at entertaining while educating the customer. Children and youngsters can have good time in addition to learn/remember words by seeing, reading and typing them. It helps the user mastering fast and correct typing skills. It may help the player increase their concentration by focusing on doing one task (writing particular words correctly). It can be a way for people gathering and socializing while playing this game. Friends or family members can try to compete with each other typing and enjoying the game. Meaning, it brings people together. The game works as follows, a player will try to shoot random words and achieve highest possible score. The game is in a wave-mode, meaning, after shooting the first

wave of words that comes out of the top of the computer, the next wave comes after a little resting break. The next wave will be more difficult either by choosing longer words, faster movement of words, words that are not used in normal conversations or all together. This will make it highly challenging. Our fingers will be trained to write faster and it also improves the brain recognition of words. The player sees their score at the break screen after each wave. This gives the player some motivation and keeps them interested in continuing the game. The game is simple, does not need any saves at any point during the game and can be exited peacefully. Even better than card games (e.g. solitaire) in this context where the player has to complete the game. Here, you can start, play, exit or resume. This game is based on existing products but is more advanced. It's a game, which can be compatible on all platforms and devices (can be played on tablet, mobile, pc, etc). It does not require a loading time, high specs or internet to enjoy it. Everyone can play it at anytime and anywhere. We are a group of four students, who have this project as part of Software Development 2, C++ course. The time-frame to deliver a complete product is approximately 70 days. The four of us will participate in the development of the game. It is assumed that each one will work approximately 8 hours per week. The product will not require us (developers) as students spending money to achieve the goal. The only cost would be the software licenses (overleaf, Clion, etc.) we received from university.

1.2 Similar Products

This product is inspired by ZTYPE, a type to shoot game[7] developed by [5]. This game combines shooting and typing. The attractive qualities of this game is the shootings, type fast to survive challenge and do that while learning vocabulary. This can be considered educational and enjoyable at the same time. There are quite many tutorials on internet that teaches how to type faster, but when it comes

to learning things fast, it is always better to make the teaching learning process very enjoyable. This way, what we learn will stay longer in our memory and we understand it faster and better. This is one of the reasons, why the type to shoot game is very important. It is very entertaining and suspenseful. it makes you want to go further with the levels, no matter how many times you failed. Games such as, Guitar Type, Speed Typing, Typing Racer, Type rush, etc. These are some of the similar products to our game. They have similar functionalities with different presentation.



Figure 1: ZTYPE Type to shoot

2 Process

There might have been one question that the project faced since the day it started off. "Where should we start and how should we develop it?" The discussion and explanation below attempts to the process and progression the project took in through out the development process.

2.1 Preparation

Our project period has been approximately 10 weeks. Before that, we sat together a couple of times to sign the group contract, discuss what the project is going to be about, and to make the product vision. The group contract is about meeting commitment and mutual respect. We gathered a few times to decide what the project should be about. We did not have a lot of knowledge about embedded systems. Hence, it was clear for us that it is going to be about game development. At the beginning we were unsure what game we should choose to work on. The suggestions were to make a game similar to one of the following:

- Breakout
- Zelda
- Mario
- ZTYPE

We made a vote, and the unanimous decision was to develop a game like ZTYPE. ZTYPE is a web-based game, where a spacecraft shoots waves of words coming down from the top of the screen depending on users input. We then wrote the product vision and got the approval from our supervisor Christian Auby.

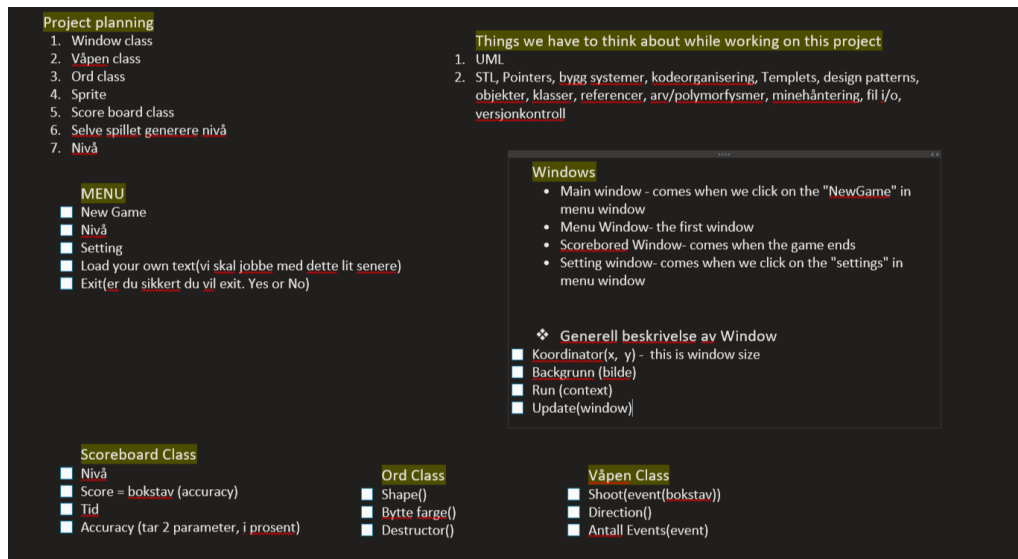


Figure 2: Dividing the product implementation in modules

2.2 Meetings

During the ten weeks, we had two mandatory meetings each week. One of which is a group meeting. We used to hold it on Monday mornings at university. At these meetings we reviewed what we have done so far, helped each other whenever we found ourselves stuck on our tasks, assigned tasks to everyone, merged branches together, and made a plan about, what we should come up with for the coming week, with the rest of the time remaining. The other meeting day was every Thursday, where we have a meeting with our supervisor from 13:00 to 13:30 at his office. At every meeting we showed him the progress we made since the last meeting, asked him questions related to code, git, report and structure, and he gave us good help and motivation to keep us going. We used to gather for about two hours before the meetings with Christian Auby to note ideas and collect questions to discuss and take up in the meetings. Other than this days, we all had our own separate plans, where we worked on our separate tasks.

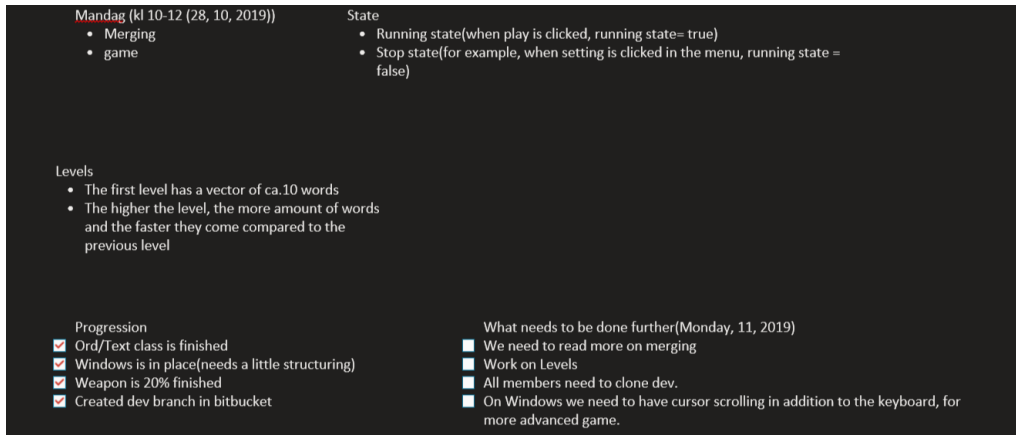


Figure 3: Discussed points during one of the meeting days

2.3 Tools

We have used multiple of tools. Some of them are paid by the school, others are not. We got licenses for the paid ones from university. They are Jira, Bitbucket, Clion, and Overleaf. The free ones are Git, Microsoft Teams, Google docs, social medias such as Facebook Messenger, etc.

2.3.1 Jira

Jira is a proprietary issue tacking product developed by Atlassian that allows bug tracking and agile project management[jira]. We used it to create and assign different issues/tasks to our group members. We used its Time-sheets feature as well to log time and count the hours for each group member which they used working on the project.

2.3.2 Bitbucket

Bitbucket is a web-based version control repository hosting service owned by Atlassian for source code and development projects that use Git or other distributed

version control systems [1]. Bitbucket is used to bring all the members together. Meaning, it helps us work separately in our own branches and merge our code together with the help of Git. Thus, we all get to combine the codes we worked on and share them via bitbucket and work together.

2.3.3 Overleaf

Overleaf (ShareLaTeX previously) which is an online Latex editor that allows real-time collaboration and online compiling of projects to PDF format[2]. We used Overleaf to write our project report in a co-operative way.

2.3.4 Clion

Clion is a cross-platform C and C++ IDE integrated with the CMake build system[3]. All of us used this IDE editor to write our source code in C++. It was recommended to use by the supervisor, and we have used it in the course before this project and felt it was a good option.

2.3.5 Git

Git is a distributed version-control system for tracking changes in source code during software development[4]. We used it to manage our code (push to server, pull from server, update code, merge code pieces), and communicate with Bitbucket server.

2.4 Libraries

SFML (Simple and Fast Multimedia Library) is the only library we used in our project. It is a cross-platform software development library designed to provide a simple API to various multimedia components in computers. This library was recommended by our supervisor. So, in the preparation time, we downloaded SFML from its website. To set up the library we used the tutorial that Christian Auby

provided us on Canvas. We got SFML ready on each of our pc and tested the library in the preparation period (before the project started). Some of the features we used in SFML are:

- Sprites and Textures: to draw different objects such as weapon, background, mute button and bullet.
- Text and Font: to write and draw the words falling and move them.
- Music: to play different sounds in different situations such as, background music and bullet sound.
- Windows: To create different windows such as menu window, game window and settings window.

The SFML library is a big, comprehensive, and easy to learn and apply library. Therefore, we did not need to use other libraries, SFML provided us with all the utilities we needed.

2.5 Version control

Git was used as the version control for the project. It is also widely used in coding courses at University of Agder. This tool made our collaboration easier, by allowing us to make our code available to other group members. The server we used was Bitbucket, which is a server distributed by UIA. Git can be used to merge different code pieces into one. It is as well a good and secure storage space for our code. All group members have used it before. Git has many functionalities and it was necessary for all group members to understand it very well. Therefore, we used some time organizing the branches on Bitbucket, learning about merging and different Git commands, and helping those of us who needed help. In addition to that, we also received help from the supervisor and student assistance on this topic. On Bitbucket

we had 6 branches. Four feature branches, one branch for each of us to push our code to. This code could contain bugs and errors and is very often updated. All of us had access to all feature branches, which allowed us to pull code from each other, help each other on things we are stuck on, and write code that gives less bugs while merging branches. A main branch called “Dev”, where we push the working, bug-free, and tested code. All group members have access to this branch. The code there gets updated after adding a whole feature to the game or finishing a big task. And an origin master branch, where Only one member has access to this branch, and when we have a clean, stable and complete code, this master branch gets updated.

2.6 Gameplay

2.6.1 Window overview

When running the game, a window that consists a few options appear. the player then get to choose among these three options, “New game”, “Settings”, and “Exit game”. Choosing new game will take the player to the game window where he/she can start playing the game. Settings button leads to a settings window where the player can adjust the volume in the game, turn up, turn down, or mute the sound. “Exit game” button will shut down the entire game.

2.6.2 Game logic

The game we are developing is not innovative. The original game is called ZTYPE and is a close-source, web-based typing game. The game is about waves of words coming down from the top of the window, and a spacecraft positioned at the center bottom of the window shooting the words. The player shoots the words by writing/typing them on the keyboard, one after the other. When shooting a word, after typing its first character, it gets marked with a different color from the rest of the words, meaning, when words come down they are red in color, but when typing the

first later of one of the words, the word changes color to green, this helps to show which word is currently being typed and the player must complete typing the rest of the characters of that specific word before starting with another word. Each correct character typed releases a rocket from the spacecraft towards the word. When typing a wrong character, no rocket is released, and the player gets notified about it by blinking the spacecraft. The words come in form of waves. Each wave has a number of words, and its words have a determined speed. Each wave is more difficult than the previous one. The difficulty is determined by increasing the number of words in the wave, or moving the words in a higher speed, or both options. When finishing a wave, a scoreboard window is displayed showing the player his/her score, before the next wave comes. The player loses if at least one word come down all the way to the bottom of the screen or touches the spacecraft(weapon). When the player loses, a scoreboard window is displayed again showing his/her final score and giving the chance to start over again. There is a file containing hundreds of words. This file is used as default to load the words from it. We wanted to develop a function “Load your own text” allowing the player to load his/her own text file (.txt file), where each line in the file becomes a wave of words. The file must exist locally on the pc.

2.7 MVP (Minimal viable product)

After choosing what type of game to make, the next step was to figure out how to build it. First, We divided the game into different tasks and each member took in it and decided which of these tasks should be prioritized. Minimal viable product helps us configuring what to focus on more and which part we shouldn't spend much time working on. There were three priority categories:

- Must: these are tasks we MUST include, in order for the game to work at its minimal.

- Should: these are tasks that makes the game user friendly and fun to play.
- Could: these includes additional tasks that adds extra functions and make the game more advanced and interesting.

Must	Should	Could
Game window	Main and setting window	Loading own text file
Waves of words	Spacecraft and rockets	Moving background
Implementing wave difficulty	Implementing scoreboard	Showing player stats
Automatically moving words	Music and audio	Eliminate wave button
Detecting correct input	Adjusting music	
Isolate target (word)	Background image	
Ignore wrong input	Stop and resume game	
Exit game	Restart game	
	Dynamic level generating	

Table 1: Minimal Viable product

2.8 Division Of Labor

We agreed to appoint Filmon to be our group leader. He planned the meetings, booked group study rooms, and delegated tasks among the members in the group. Tasks were assigned in group meetings and confirmed using Jira. Mohammed had most experience with Git in the group and we gave him the responsibility for the master branch on Bitbucket. The main tasks were the following:

- Words: drawing, moving, isolating and deleting the words.
- Windows: designing menu, settings, level and game windows.
- Storage: generating words and sorting them in waves and levels.

- State machine: changes between states after receiving a signal.
- Game manager: game system that manages objects and handles events.
- Entity: a class where we created an object from words.
- Media: background music and image and adjusting sounds.
- Scoreboard: implementing scores and showing results.
- Shooting: the process of moving the bullet smoothly from the weapon to a word.
- Report: Writing different sections of the report.

These tasks were divided and assigned among group members. Each member have worked on their own tasks. They have worked efficiently and all members were helpful in assisting each other on things they got stack on.

3 Product

This product contains different functions from SFML libraries, windows, objects, features, patterns, etc. Each of the objects and functions will be discussed further in this section.

3.1 Main menu

once the game started, the first window shows the menus. The first menu option is the "New game" option. If this option is clicked, it takes the users to the next window, where they can play the game. As soon as the game begins, words will start flowing down and the background music starts playing. Hence, the player can start playing the game and enjoy their time.

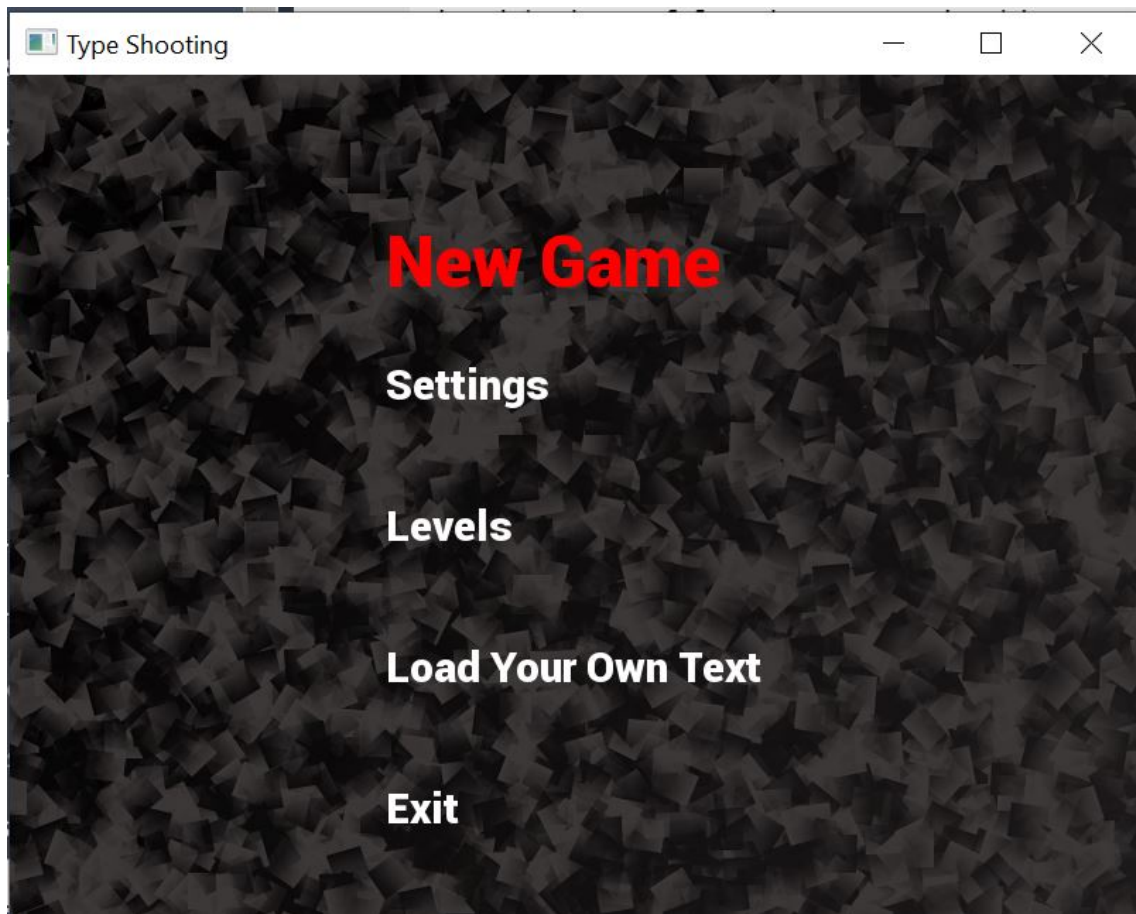


Figure 4: menu window

3.2 Weapon

The weapon was in the early stage of the game development. The Weapon is placed on the bottom of the screen. It is not a movable object that means it is stationary. The game will end when waves collide with Weapon or the ground.

3.3 Bullet

Bullet is one of the main characters in the game. It is found at bottom of the screen inside the Weapon. It is created when a user type the right character. once it

leaves the weapon, it immediately moves towards the wave of words that are coming down and hit the typed character making it disappear in the process. When the Bullet collide with the words, a new Bullet will be created in the weapon ready to be fired and the damaged or used bullet will be removed. Bullet is stored in vector and removed from vector when collision happens. figure 5 shows how Bullet moves towards the wave of words. Waves return the current position and then the Bullet will be directed on a specific target. Many Bullets can appear on the screen at the same time if a user types fast.

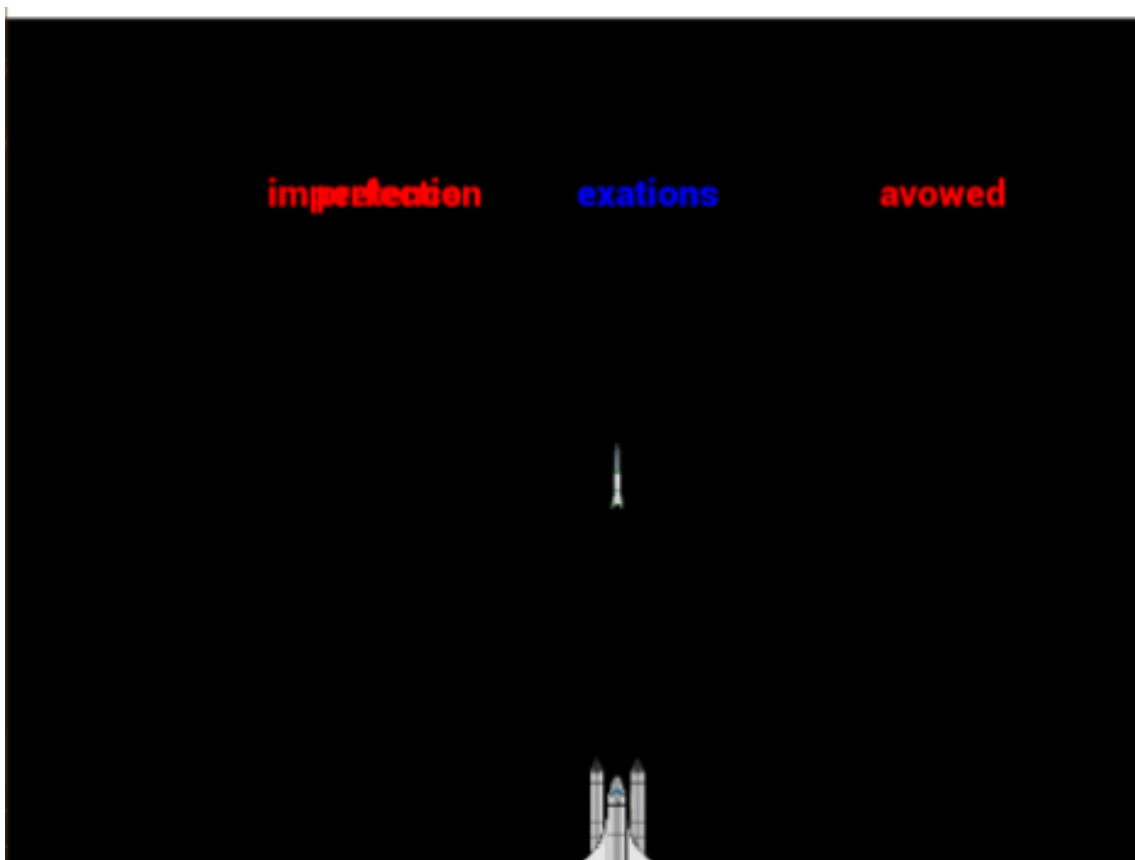


Figure 5: movement of bullet

3.4 words

There is a file that contains Words. Words are read from a file and created to be a moving entity. Waves of randomly chosen words move towards the weapon or the ground in every level. They are the main characters of this game.



Figure 6: Enemies spawning

3.5 Storage

The game has a file with thousands of words in it. For it to function it is necessary to have access to the list of words and create entities from them on demand. this is also very useful when a player loads their own text.

3.6 Scoreboard

Scoreboard was one of the objects that the game should contain as agreed by the group. Two scoreboards were designed. The first Scoreboard is displayed after each level is finished. It shows the score that the player achieved and the number of the upcoming level. The score is calculated depending on how many characters the player managed to shoot. This scoreboard will get displayed for a specific amount of time and then it disappears. When it disappears, the next level starts. The other scoreboard is displayed when the player loses the game. It will show the players their final score and provide them the options to either restart a new game or exit the game. It will direct the players depending on their choice to either start a new game or close the game.

4 Implementation

This section explains how the different components are actually implemented and why they are chosen to be as such in the first place. The architecture of the game can be roughly divided into Objects and managers, as the implementation is more or less an object oriented programming. Objects in this case is a single Entity which can be uniquely identified. An object is composed of one or more components and can have various behaviors (methods). A component is a datatype, for example int or Boolean. Figure 7 shows simple architecture of objects in the game.

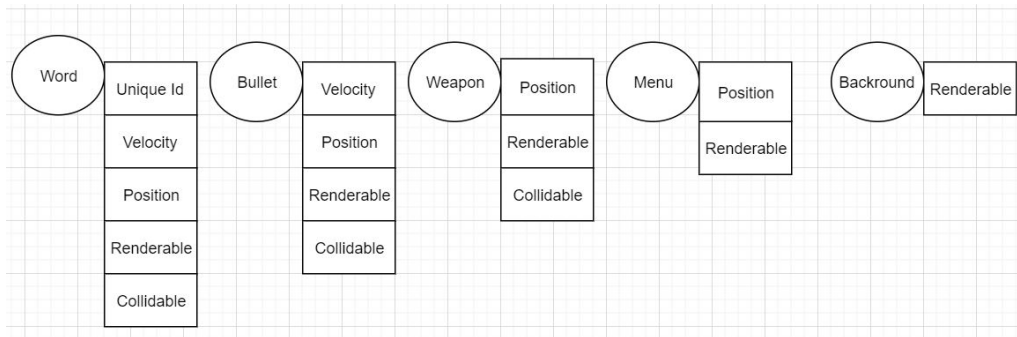


Figure 7: Game components and their most important properties

Managers are the brain of this game. They are systems that coordinates interaction between the various objects, states and events. There are two main manager in this product, a game manager and a menu manager. Discussion and explanation about how classes are applied and come to be a complete application.

4.1 General Structure

The general structure, however simple, is designed such that the game performs its operations effectively. The majority of the classes are grouped in to two abstract classes, the object class and the state class. The majority of classes inherit from these two classes. Inheritance generally dominant design pattern between the classes, in

4.2 Game State

```
enum class gameState { MainMenu,NewGame,Setting,Exit };
```

Listing 1: state

our game has simple states which make it easy to switch between states. When a game starts, the Main Menu displays, then the user can choose one of the options. If the user choose newgame, the current state is set to newgame, which then runs the game. If the user loses, scoreboard pop up and give the user option to restart the game or exit. The user returns to Main menu to proceed from there. figure 9 illustrates how game state works.

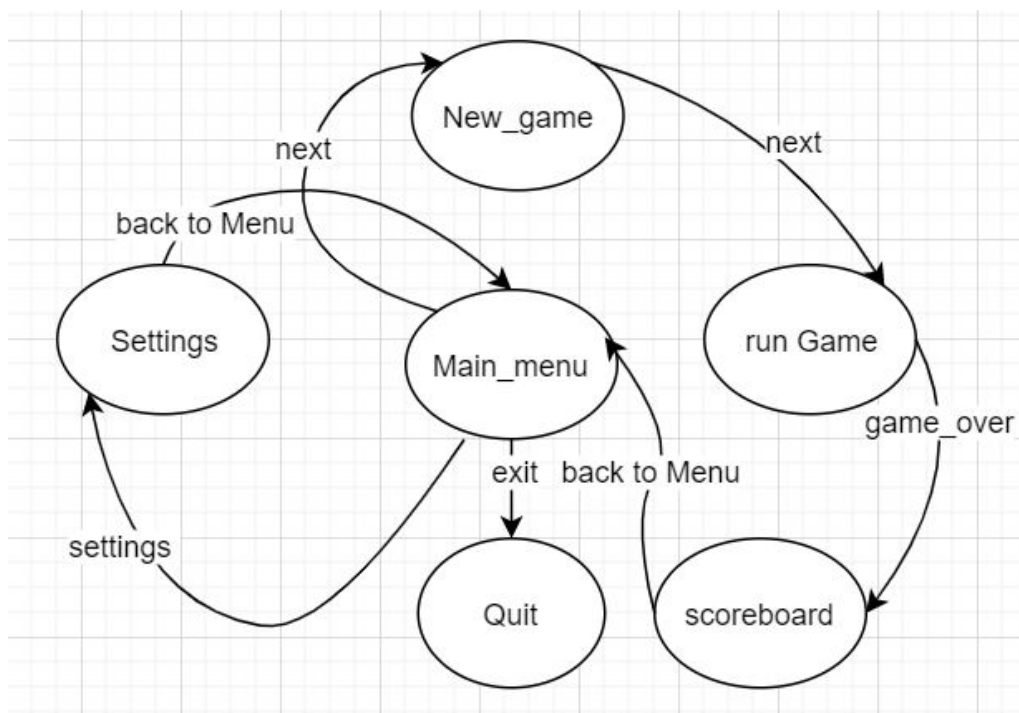


Figure 9: Game States

4.3 Class Object

Object is an abstract class. This class declares an object as something that has specific size, form, position in (x, y) plane and that can be drawn on a window by the only method, draw method, it incorporates. Weapon, bullet, Entity and Entity classes inherit from this base class. A UML representation can be found at figure 10.

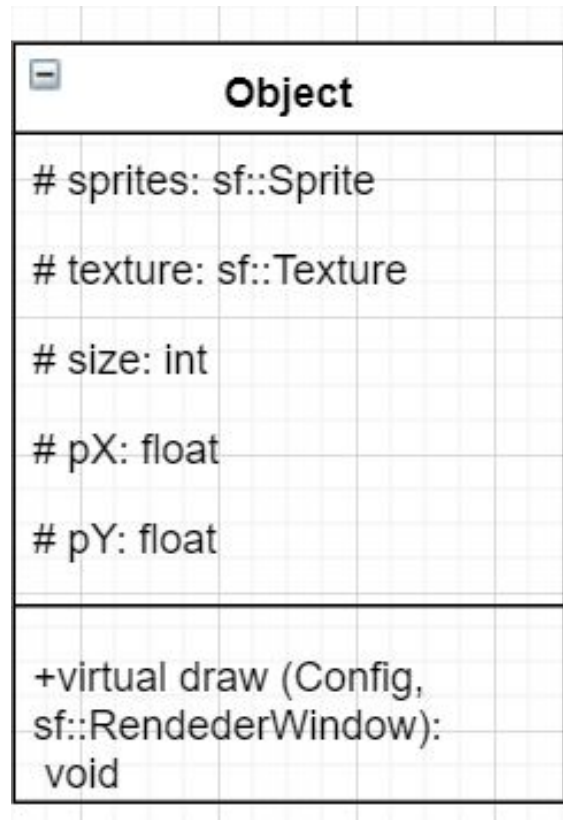


Figure 10: UML representation of Class Object

4.3.1 Class Entity

In this game, words are considered as enemies. Class Entity represents enemies. Entity class inherits from Object class, and is implemented by another class named

EntityProxy. Figure 11 shows relationship between Object, Entity, EntityProxy.

This class creates a word entity with a unique id. this entity has the ability to move, to delete an alphabet and other behaviors that can be manipulated, thus, change the entities composition.

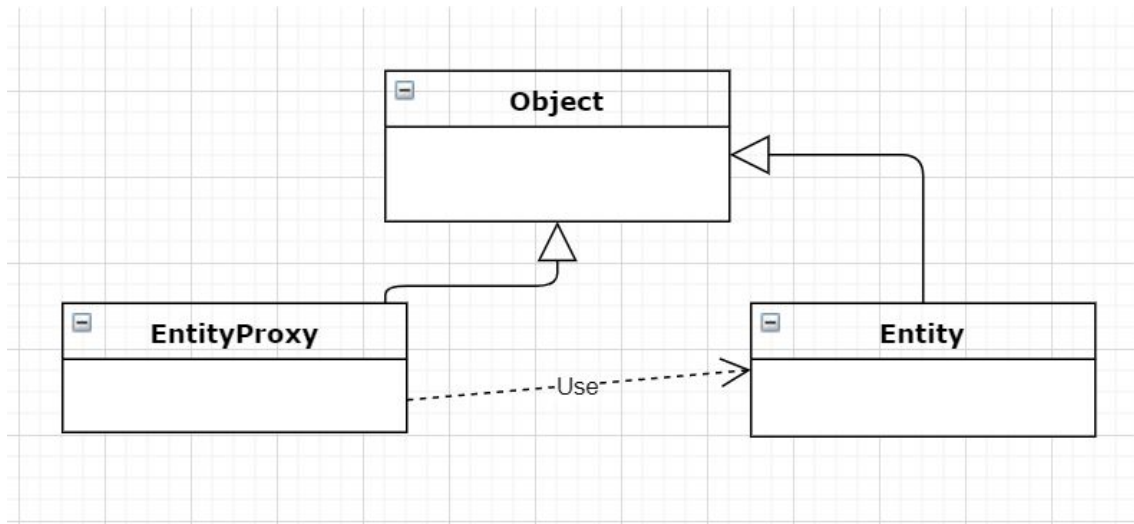


Figure 11: Object-Entity-EntityProxy relationship

This class is vital part of this game. Therefore, the completeness of the game totally depended on correct implementation of this Entity class. Some of the challenges this class have to deal with is to dynamically draw words as a wave of some words in every level, deleting correctly typed alphabet and how to move the words towards shooter.

4.3.2 Waves of words

To solve the problem of drawing words as waves, three active words are added to a vector before they are drawn on the screen, then loop through the vector to draw them. Every 7 seconds, the vector adds three more words and draw them on the screen, thus waves of three words at a time are drawn every 7 seconds in every level.

4.3.3 Isolate target (word)

Another challenge that took some time is setting up unique target for the shooter. The logic used here is to check if the word starts by the typed alphabet, however, most of the time multiple words start by the same alphabet either because they originally start by that alphabet or after deletion of an alphabet. The strategy chosen to deal with this mess is, to create two arrays and an int variable. one of the arrays store the Id's of the active words, and the second array is called priority array and it stores active words that currently starts with the same alphabet and the variable holds the id of target.

The priority array plays an essential role, setting up a target after the current target is destroyed and there are currently multiple words that start by the same alphabet. It takes the smallest id and make the entity with that id to be the target.

4.3.4 Moving Entity towards the object

A word entity should move towards the position of the shooter. Words get a random position at the screen, therefore, they must somehow figure out which direction to take, to move towards the shooter. To be able to move towards the shooter, the entity must have a velocity. The vector between entity and shooter is calculated by subtracting their positions and then normalizing it as shown in listing 2.

```
Vector2<float> GameManager::normalize(Vector2<float> dir) {  
    float hyp = sqrt(direction.x * direction.x + direction.y * direction.y);  
    dir.x /= hyp;  
    dir.y /= hyp;  
    return dir;  
}
```

Listing 2: Deleting alphabet implementation

4.3.5 alphabet deletion in Entity class

The most important behaviour of an entity is to erase one alphabet at a time by the request of the game manager and resize itself. When a player correctly types the first alphabet of a targeted word, the game manager asks the target to delete that alphabet and resize itself. Events like key typed, are handled by the event handler of SFML library. The deletion of alphabets however, are handled by the Entity class. The block code below shows how deleting killed alphabet is implemented.

```
void Entity::erase_killedChar(char t, Bullet &bullet) {
size_t found = this->getWord()->getString().find(t);
try {
    if (found == 0) {
        str2.erase(0, 1);
        if (str2.empty()) {
            status = false;
            active = false;
        }
        this->getWord()->setString(str2);
    }
}
}
```

Listing 3: Deleting alphabet implementation

4.3.6 EntityProxy class

EntityProxy class acts as interface to access the class Entity. This class inherits from class Object and implements every method in Entity. By using a pointer of type Entity, this class can access and operate on class Entity.

4.4 Finding Direction

Words and bullets need direction to go in a specific path. They need some type of pattern. This section, will focus on discussing why directions are needed and how they can be implemented. There are two movable characters in the game. One is wave of the words and the other is Bullet.

4.4.1 direction of Bullet

As mentioned above the Bullet is a movable object and it is located at the bottom of the screen inside the weapon. It is going to be fired towards the wave of words, when the right character of the word is typed. To move the Bullet towards the direction of waves, Bullet needs a direction where the target is located. Words move downwards and they take different directions relative to their starting point. They should be found in random positions, at the top of the screen. The words return to their position each time they change the position and return the current position at the same time, which makes it possible to find the current position of words. Before finding the direction, we need the distance between the words and Bullets. Distance "d" between words and Bullets is computed with the help of Pythagoras theorem as illustrated in listing 4.

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

```
bullet_position = sf::Vector2f(bullet_sprite.getPosition().x, bullet_sprite.getPosition().y)
wave_currentPositon = sf::Vector2f(waves->getPosition().x, waves->getPosition().y);
distance = bullet_position - wave_currentPositon;
```

Listing 4: distance

Bullet must move in the right direction to the target, which is calculated using

a mathematical formula for a unit vector as shown in listing 5. The mathematical formula for a unit vector is

$$\hat{\mathbf{a}} = \frac{\mathbf{a}}{\|\mathbf{a}\|}$$

```
double sqrt1 = sqrt(pow(distance.x, 2) + pow(distance.y, 2));  
mainDirection = sf::Vector2f(distance.x / sqrt1, distance.y / sqrt1);
```

Listing 5: direction

Listing 5 illustrates how the Bullet actually finds the direction and moves toward the specific target. On the way to the target, Perhaps the target changes the position, therefore, the Bullet has to update its position each time until it meets the wave. Without direction, the Bullet could not move in the right direction, that is why we needed a computed direction.

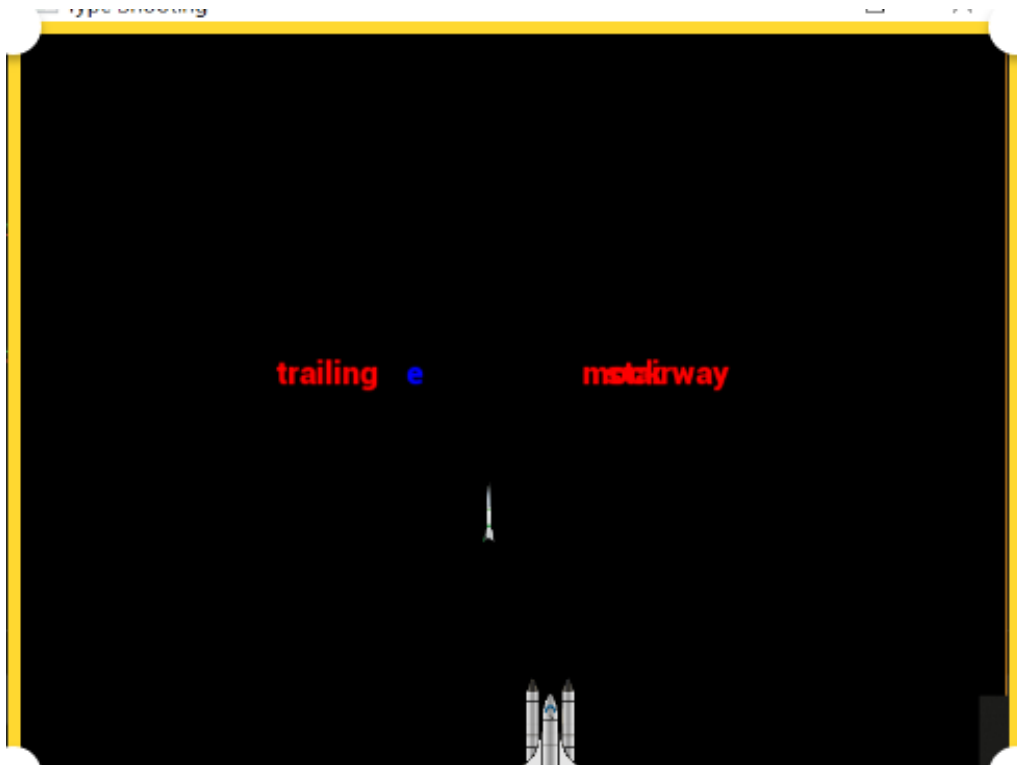


Figure 12: Type shooting game Object

4.5 Event

The final missing piece required to tie everything together are events. Each process decides how to react when it receives some sort of an event. The following are some examples on how and why it is useful to have events in this game.

- If the player types on the keyboard(an input Event) while the game is running, then the input system will receive this event and check what to do.
- When the system receives the Event, it first checks if the input is correct or not, if it is correct, then it deletes the first alphabet and shooting will occur instantly.

The other scenario can be when the collision happens

- If the player shoots the wave of words that is coming downwards, and then collision occurred, the system will detect the Event and check what to do. Once it checked this event, the Bullet will be removed from the vector.
- The event can also be useful while the game is not running. The key pressed indicates that the state changed.

4.6 Class Storage

Storage class is a container for enemies, in this case, words, and it makes spawning, tracking and dynamically loading levels easy. As shown in fig 13 EntityProxy is associated with storage class as a parameter of containers.

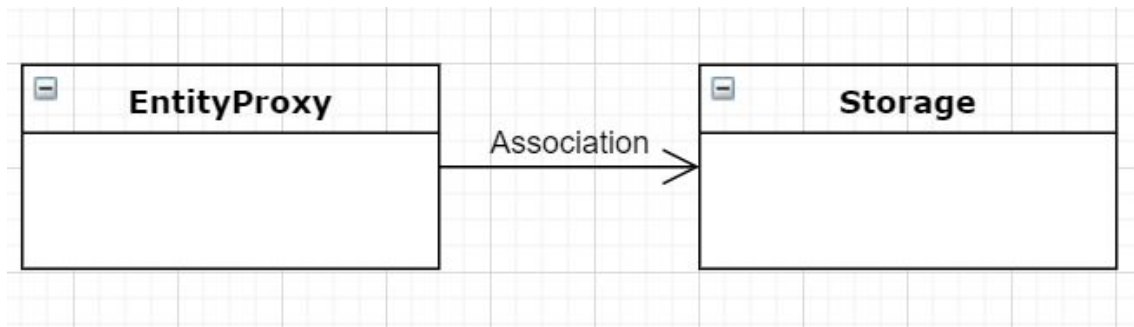


Figure 13: Storage - EntityProxy Association

4.6.1 Creating dynamic levels

Levels are created here. When the application initially starts the game, it reads thousands of words from a file and store them in a container. It loads level 1 to level three. If only two loaded levels remaining to be played, then game Manager asks storage to load three more. The ability to load levels dynamically is an achievement.

4.7 GameManager class

This class contains the game loop itself. In this class, the game runs, objects are rendered, levels are loaded, change in levels happen and events are handled. This is the motor that keeps the game running and the system that coordinates interaction between the different renderable object classes. Some of the most vital implementations that are implemented in this game are the following:

- Vector normalizing
- Choosing a target
- Changing level after each level is successfully completed.
- Resetting state variables after level change
- Collision detection

4.8 MenuManager class

One way of making the game interactive is to have windows that can handle events. We have created several windows that contains different features. The menu manager inherits from state, therefore, it implements the go next method in state, that sends a signal to the machine which state to run. The main function of game manager is to act as the system that accepts input from the menu buttons and sends a signal to the machine which state to run next.

This class contains four methods to navigate up and down, one method to handle events. The buttons are implemented as an array and are allocated a specific position on the window.

In the window header we have chosen to use array, thus, we set each index of the array to different functions. For example, `setFont(font)`, this makes it use the font that is assigned to it. `setCharacterSize(50)` was used to set the amount of character

the option should contain, which is 50 in this case. `setString("new game")` is used to set name of this option. `setFillColor(Color::Red)` was used to put the color of the current option which is about to get clicked on as red, when the cursor moves to another option, former options change color to white. `setPosition(Vector2f(width/3, height/(MAX_NUMBER_OF_ITEMS + 1)*1));` //this is a formula to position the options right at the center. The `*1` implies the first or the "new game" option and it will come before the rest of the options. It gets higher as the option (or array) number increases.

This had been applied for all the option arrays by changing the array index number for example, `option[1]`, `option[2]` and so on until we fill our array list, `MAX_NUMBER_OF_ITEMS`, which has place for 5 elements. The rest of the options are settings, level, load your own text, and exit.

The window and the keyboard have been implemented as follows. The keyboard's up and down arrows, can be used to go over the list of menus(options).

```

while (window.isOpen()) {
    while (window.pollEvent(event)) {
        switch (event.type) {
            case sf::Event::Closed:
                window.close();
            case sf::Event::KeyPressed:
                if (event.key.code == sf::Keyboard::Up) {
                    MoveUp();
                }
                if (event.key.code == sf::Keyboard::Down) {
                    MoveDown();
                }
                if (event.key.code == sf::Keyboard::Enter) { //when Enter is pressed
                    if (clickedMenuIndex==4){
                        window.close();
                        context.SetRunning(machine_running);
                    }
                    else if (clickedMenuIndex==3){
                        context.SetState(Machine::StateIndex::ScoreBord);
                    }
                    else if (clickedMenuIndex==2){
                    }
                    else if (clickedMenuIndex==1){
                        window.close();
                        context.SetState(Machine::StateIndex::Setting);
                    }
                    else if (clickedMenuIndex==0){
                        window.close();
                        context.SetState(Machine::StateIndex::NewGame);
                    }
                }
            }
        }
    }
}

```

Listing 6: menu window implementation

5 Testing and validating

Due to lack of enough time to make unique tests for each class, methods and functions we created, our tests have been quite general for the whole product. Our testing was done by running the game and checking different things we have implemented. These tests have been applied multiple times during the developing process. The tests mentioned in the table below were used to test the product and to verify that things worked as they should. Each test has an indicator whether it passed or not.

Tests	Passed
All main menu items work as they should	Yes
Words moving downwards in horizontal and vertical direction	Yes
Words start flowing downwards from the top of the screen at random diagonal position	Yes
Correctly translate from keyboard key code to ascii code	Yes
Listening to user input, take the correct one and ignore the wrong one	Yes
For each correct input from keyboard, bullet is released towards the correct word	Yes
Shooting first character of a word will isolate it and change its color	Yes
When a word is isolated, no other words can be shot until the current word disappears	Yes
When a word is completely typed, it is possible to shoot any other available word	Yes
Display scoreboard after each level and also when player loses the game (Game over)	Yes
Generating dynamic levels after the first three static ones	Yes
Mute volume	Yes
Adjusting volume	No

Table 2: Test and their results

There are only two bugs we came across during our game development process. These are:

- We use many windows instead of using one window. We use a window for main menu, another one for settings and another one for game, instead of rendering to only one window. Scoreboard pops up as an own second window when it gets displayed.
- Adjusting volume was not implemented correctly. In settings window the player gets the chance to turn up or down the volume. In our product this feature does not work completely.

6 Discussion

As mentioned above, the assignment in this project is to build and deliver a complete software written in C++. With 4 up to 6 developers allowed in a team and 10 weeks until the date it should be delivered, the development process of the game started off. Majority of the goals set at the start er met, thus, this product satisfies a complete product with minimum requirements. The group is satisfied by the result of the process. However, there were and still are some challenges and failures. The sections and subsection below will attempt to address these issues in detail.

6.1 Process

As the team came to understand through out the development process, good planning, timing and prioritizing tasks, some of the main factors in a project. Though there is overall satisfaction among the members, the above mentioned factors were not followed carefully and as a result it impacted the product negatively.

6.2 Git

For version control, the team used git. git presented unparalleled challenge, and it might be the challenge that took most time in the process. It has several command terms and the group faced quite a challenge to implement them. From the groups perspective, git deserve its own course.

6.3 Product

Must	Should	Could
Game window	Main and setting window	Loading own text file
Waves of words	Weapon and bullets	Moving background
Implementing wave difficulty	Implementing scoreboard	Showing player stats
Automatically moving words	Music and audio	Eliminate wave button
Detecting correct input	Adjusting music	Explosion animation
Isolate target (word)	Background image	
Ignore wrong input	Stop and resume game	
Exit game	Restart game	
Basic three levels	Dynamic level generating	

Table 3: Strike-through shows implemented features

The product have most of the components it was intended to have. Most of the components and functionalities the game have and their implementation is explained in detail in implementation section. Since it would mean repetition to discuss on those implemented successfully, the discussion below stresses on features that are not implemented.

6.4 Design and multimedia

In our product we used Text from SFML library to menu items and clickable objects. These direct the player from one window to another when he/she hit enter on the Text. It changes color when it is focused and get underlined when it is clicked. We could improve this by designing clickable buttons which could make our menu look better. Buttons are more interacting objects than text and function better when hovering and clicking. We used free licensed images from the internet as

our background images. We could design images which best fit our game using Photoshop or other picture editing programs to fit our game. We could as well design a logo for the game. Our color use could be better. Word colors and menu item colors could be chosen to fit the background images and the game genre. The background music and sounds we used were fetched from the internet. They could be chosen more carefully to fit our game. We could have as well used sounds in other places such as, when clicking on a menu item or on collisions (between word and bullet).

6.5 Words overlapping

When the words come down from the top of the screen, they start moving from random vertical position. This allows two or more words to start at the same position. We did not implement a way to prevent this from happening. In addition to that, while words are moving, some from left, some from right, towards the weapon (middle bottom of screen), they can cross and overlap each other. This can make the overlapping words difficult to read and hence, to type. Here we could have added a function or method to avoid such situation.

6.6 Rotating of bullet and weapon

In the shooting animation we have, when the bullet is shot, the image of the bullet and the image of the weapon firing it does not rotate or change direction to point at the word getting shot. This should have been fixed using some mathematical formulas in geometry and trigonometry for rotation according to a given position in (x,y) plane. For us this was not a priority and due to time shortage, we could not apply this rotation in our product, but it could be considered as an update for a later version in the future.

6.7 Showing score and status

We were able to implement scoreboard that shows players score and their upcoming level as described in a previous section. One more feature that is nice to have is players status. It would give the player his/her statistics, which describes how their progress has gone along their play and what their record score is. This feature has not been added to the product due to time shortage and its low priority on our tasks list.

6.8 Challenges

While working on this game, the group has faced a few challenges. We managed to solve most of these challenges by discussing and getting help from the supervisor and student assistance in school.

6.8.1 Shooting movement

Regarding the animation of shooting words, We could move the bullet from the weapon to the specific word, but the movement was like jumping from one place to another. The challenging part was to smoothly move the bullet in the desired direction. We made a lot of exploration on the internet and saw several tutorial videos on YouTube, we could not understand it completely. We then asked our supervisor about it, who helped us with figuring it out. It turned out that we had a little wrong understanding about the smooth movement in coding. At the beginning, we thought it was only about writing a piece of code. But Christian Auby explained to us how mathematics and physics are involved here as well. We then refreshed our knowledge about distance and speed vectors in physics and tried to implement it with code. To get the direction, we only had to subtract the position vector of the bullet and the position vector of the word we are shooting. Here we had to apply speed as well. We decided to use fixed speed in both directions of the movement.

At the end we managed to smoothly move the bullet from the weapon to the word we are shooting.

6.8.2 Multiple Bullets

The bullet could create depends on the user's input. To move two or more bullet at the same time was a challenge because the bullet was stored in the vector, but they could only add when the vector is empty so therefore two or more bullet would not be created. Furthermore bullet from vector when the detection occurs, But Christian Auby explained that that is important to change our logic in the code. Conclusion was the to create many instances of bullet but could only be added in the vector when the player starts typing. We also managed to shoot only when the player input a correct letter if is not, the bullet should be deleted.

7 Future plans

7.1 Load and save

This functionality could be added to our game. This could require implementing user catalog, where the player defined by a name can save his/her progress to skip starting the game all over again, when they have reached some level. The saved information would be score, status and level number together with players name. The player can then load their saved progress and continue playing from there. This would make the game more enjoyable and addicting.

7.2 Network

This game could be a web-based game, where users did not need to have it locally on their pcs to play it. But could play it online as well as on the internet browser.

7.3 Pause and resume

In our product we miss the feature of pausing the game. This is always a plus to have in games. Sometimes not having this feature makes the game unattractive to players. Because having it makes the player more comfortable in playing the game knowing they can stop it at any time and then resume it right from where they paused.

7.4 Elimination of words

This feature could be added to the game to help the player when the game gets harder or they are in difficult situation. This will clear all the nearby words in a certain radius from the weapon (bottom of the screen). This feature will make the game more acceptable when it gets more difficult in higher levels.

7.5 Explosion animation

As mention at the introduction and other places in the document, the game is a type of shooting game. Thus having explosion effects may have been expected. However, for the group it was not a must feature. Hence, planned to do it at the end, if there is some time remaining. Unfortunately, the explosion animation could not be accomplished. It should be noted here that, as type to shoot game, this feature even though typical in shooting game didn't make to the must shortlist at the beginning of the game, the main reason it couldn't be implemented is not technical or lack of knowledge, rather it is a matter of priority and time.

7.6 Loading own text

One feature that could have advanced the game immensely has not been implemented. This feature is the ability to load one's own text. It was desired and attempted but couldn't be implemented. The main reason is time and difficulty to find a library, because c++ does not have easy way of accessing system directory. Though whatever the reason, the failure to implement this particular feature is accepted as the most sever implementation failure by the group.

8 conclusion

The document began by introducing a game development project, written in C++ as a final project for DAT220-G19H, software development-2. The journey started with out enough knowledge in game development, time there fore the goal was to deliver a game that must have certain components and additional optional components that could be implemented if time and our programming skills allows it.

The process went almost as planned. Even with some ups and downs, a game that accomplishes the goals, that were sett at the beginning are met.

There are, however, some goals that did not came to completion, even though they don't impede the project from being a success. One of them being the "load your own text" feature.

All in all, the game does fulfill the minimum requirement it needed, and assessed as a success by the group.

9 Individual Report

9.1 Filmon Berhe Mebrahtom

My name is Filmon Berhe Mebrahtom, a member of group 7 in game development project for DAT220-G19H, a course in software development-2 at University of Agder. My team developed a type to shoot game, and me as a member participated actively in the development process. One of my roles in the team was to lead the group. As a leader of the team I took responsibilities to coordinate meetings, development planning, task assignment and group discussion. In addition to my role as a leader, like every member of the team, I was assigned tasks to implement. I implemented entity class, storage class, game manager class, and dynamic levels (as part of storage class). In addition to the mentioned classes, I was also supposed to implement a class that could enable a player to load own text but could not accomplish it, as the time to write report began. As a team we helped each other, and I got help from my team as I also helped in some cases. Given the time we had, I am generally satisfied in the effort I put into the project, the cooperation between the team members and the game we built. I am also thankful and satisfied by the consultation hours we had with Christian Auby, our lecturer.

9.2 Iyad Zidan

During this project I have been the one working on scoreboard and helping Mohamed with bullet and shooting process. I also worked on writing this report together with the group. I implemented scoreboard in way which is easy to integrate with my group members code. Shooting process has been challenging, therefore we were two working on it. We could find some information on internet and had to figure out other alone or with help of our supervisor. This semester was my first semester at University of Agder. Other students had experience with Git, but I have never used it before. It was a bit time consuming to learn how to use Git. In comparison to other students I have not had any course in C or Java programming. This was as well a bit of time consuming to lean myself C and C++ at the same time. I was not so good at registering the hours i worked on this project on the timesheet on Jira. However, i have learned a lot from this project, including working on a middle-sized task in a group, learning much about C++, using Git, Jira and Overleaf and not least getting some training before the bachelor's thesis. All in all, I am satisfied with our product and our teamwork. It could have been better, and we could add many other features to the game, but due to the time we had I think our product is quite good.

9.3 Mohamed Ali Abdullahi

Individual report Author : Mohamed Ali Date: 19-02-2019 Course: DAT220-G119

My name is Mohamed. I am member of group 7. I am my group were working together to develop ZTYPE game. My responsibility was to create weapon and bullet which were the most important character in the game. I created movement and shooting which was difficult at the beginning. I have been involved in different areas of our project such as states and sounds. In meanwhile I started working with states machine which was also critical our game. To manage our game in a suitable way I started implementing the states machine which switches the state back and forth. I also implemented sound manager which are meant to handle the sound of our game. I used different source to fulfill my responsibilities such YouTube video. During this project, I have gained a lot of experience both how to work with team and big project.

9.4 Yeronis Assefa Hubena

Individual report. Date: 19/12/2019. Subject: Software Development 2 (C++). My name is Yeronis. I am a member of Group 7 in the C++ Software Development-2 subject. As a group, we have chosen a game development and decided to develop a typing game called ZTYPE. All members have their own roles while working on this project. My role was to create windows, images and sounds. My First role was to make the windows class. As the window is the base for the whole game, I needed to start as soon as possible and spend more time on it. I made the menu window, settings window, levels window, load you own text window. I first made about 5 options(windows), which later got minimized into three with the groups decision considering the amount of time we had. I collaborated with other members of the group to create a method to increase and decrease the volume, but the volume was not stable. This was one of the challenges we came across. But i made sure to control the sound from the main window, by muting and unmuting it. Implemented design pattern (machine state) with the help of my group members. We have worked together very well, and all of us were helpful in assisting each other. As a group, we have a potential to do better in the future, but I am gratified by the work we have accomplished for now.

References

- [1] URL: <https://en.wikipedia.org/wiki/Bitbucket>.
- [2] URL: <https://en.wikipedia.org/wiki/ShareLaTeX>.
- [3] URL: <https://en.wikipedia.org/wiki/JetBrains>.
- [4] URL: <https://en.wikipedia.org/wiki/Git>.
- [5] DOMINIC SZABLEWSKI. PhobosLab. URL: <https://phoboslab.org/>.
- [6] Typesy. URL: <https://www.typesy.com>.
- [7] ZTYPE. URL: <https://zty.pe>.